Q1

(a)

When a process is switched to kernel mode, then it will run the kernel code and access kernel data. The cache will be updated by the new data and the old data will be evicted. So when the process back to user mode, cache miss will be significant.

(b)

Yes, it can reduce the cache miss. Because the highest-level cache(like L1 cache) is private to each core. So if we have a core that only runs kernel code, then the cache miss we mentioned in (a) will be reduced. In the process that the process change to kernel mode and then back to user mode, the working set of the process will be reserved in the original core's cache.

But overall, it may not be completely beneficial. There are three reasons:

1. When there is only a little kernel code, then the core is idle and wasted.
2. When there is a lot of processes in kernel mode, then they will wait for only one core to run kernel code.
3. A process that change mode need extra overhead to change the core.

So in conclusion, it may not be completely beneficial.

Q2

(a)

1. P, user
2. mode switch
3. P, kernel
4. context switch
5. Q, kernel
6. mode switch
7. Q, user
8. mode switch
9. Q, kernel
10. context switch
11. P, kernel
12. mode switch
13. P, user

(b)

1. No. Because P is not blocked. So if not consider the timer interrupt, P will still run and no context switch will happen.
2. There is mode switch from user mode to kernel mode, and then from kernel mode to user mode. No context switch to P. P will be ready state.

Q3

(a)

- Shared properties:
  - Signal dispositions
  - Open file-descriptor table
- Distinct properties:
  - PID
  - Fork return value
  - Address space

(b)

- replaced:
  - Data
  - Entry point
  - Program image
- remained
  - PID
  - Parent PID
  - Open fds

(c)

The sub-process will be a zombie process, which is in exit state but still occupy the PCB.

Q4

(a)

(i)

P1 turnaround time is 41 ms.

- Pattern: P1 run 1ms then I/O 5ms; while P1 in I/O, P2 run until its quantum end (P1 may wait if it become Ready mid-quantum).
- P1's first 4 CPU bursts at t=0,9,18,27; each followed by I/O.
- P2 finish at 34; P1 5th CPU at 34-35, I/O until 40, last CPU 40-41.
- So turnaround = 41.

(ii)

P1 turnaround time is 31 ms. P1 each CPU burst is only 1ms, so it never use up a quantum and stay in queue 1. P2 use up 2ms in queue 1 and get demoted. When P1 come back it have higher priority and preempt P2 immediately, so after the first burst P1 only wait for its own I/O. Finish time is $61 + 55 = 31$.

(iii)

MLFQ is better for P1(41ms vs 31ms), because P1 stay in high priority and P2 fall down, so P1 wait less.

(b)

Give more tickets to the job that wait longer. Every tick, job that is not running get more tickets. After a job is chosen and run, reset its tickets. Then the job that has not run for the longest time have the most tickets, so it is more likely to be picked.